

GRAYSCALE IMAGE ANALYSIS AND ENHANCEMENT SYSTEM

Naivete Thandiwe Makobe

22244043

Abstract

This project aims to create a system that converts color images from grayscale and apply enhancement techniques. The system has grayscale conversion, contract stretching histogram equalization, gamma correction, edge detection and histogram data export implemented.

1.Introduction

This project aims to develop a program that reads PNG images and converse them to grayscale as well as apply enhancement techniques to improve the image quality and visual interpretation.

1.2 Goals

- Convert RGB color images to grayscale using three different methods
- Analyze image histograms calculating their PDF and CDF
- Enhance image contrast
- Improve the image quality through histogram equalization
- Adjust the brightness using gamma correction
- Edge detection in the image using Convolution
- Instagram data export for analysis

2. Methodology

This uses C++ to process images pixels by pixel. Enhancement analysis are done depending on choice selected by the user.

2.1 Grayscale Conversion

2.1.1 Luminosity

$$\text{Gray} = 0.2126 \times R + 0.7152 \times G + 0.0722 \times B$$

2.1.2 Average

$$\text{Gray} = (R + G + B) / 3$$

2.1.3 Desaturation

$$\text{Gray} = (\max(R, G, B) + \min(R, G, B)) / 2$$

Weights determination

Green is the brightest color according to the human eye; a red is medium while blue is the darkest luminous city is meant to create an accurate grayscale image with these weights

2.2 Contrast Stretching

Formula

$$O = (I - \min) \times 255 / (\max - \min)$$

O = Output pixel value

I = Input pixel value

Min = Minimum pixel value in range

Max = Maximum pixel value in range

2.3 Histogram Equalization

Step1: Count frequency of each pixel value (0-255)

Step2: Calculate Probability Density Function (PDF)

Step3: Calculate Cumulative Distribution Function (CDF)

Step4: Map old values to new: $\text{new} = \text{CDF}(\text{old}) \times 255$

2.4 Gamma Correction

$$O = (I / 255)^\gamma \times 255$$

Gamma Values effect

$\gamma < 1$ (e.g., 0.5)	$\gamma = 1$	$\gamma > 1$ (e.g., 1.5)
Makes image brighter	No change	Makes image darker

2.5 Edge detection

IT identifies the sharp changes in an image. This system uses a Sobel edge detector. The Sobel kernel has, G_x -detects horizontal intensity changes and G_y -vertical intensity change. Sobel is mainly used for image gradient calculations.

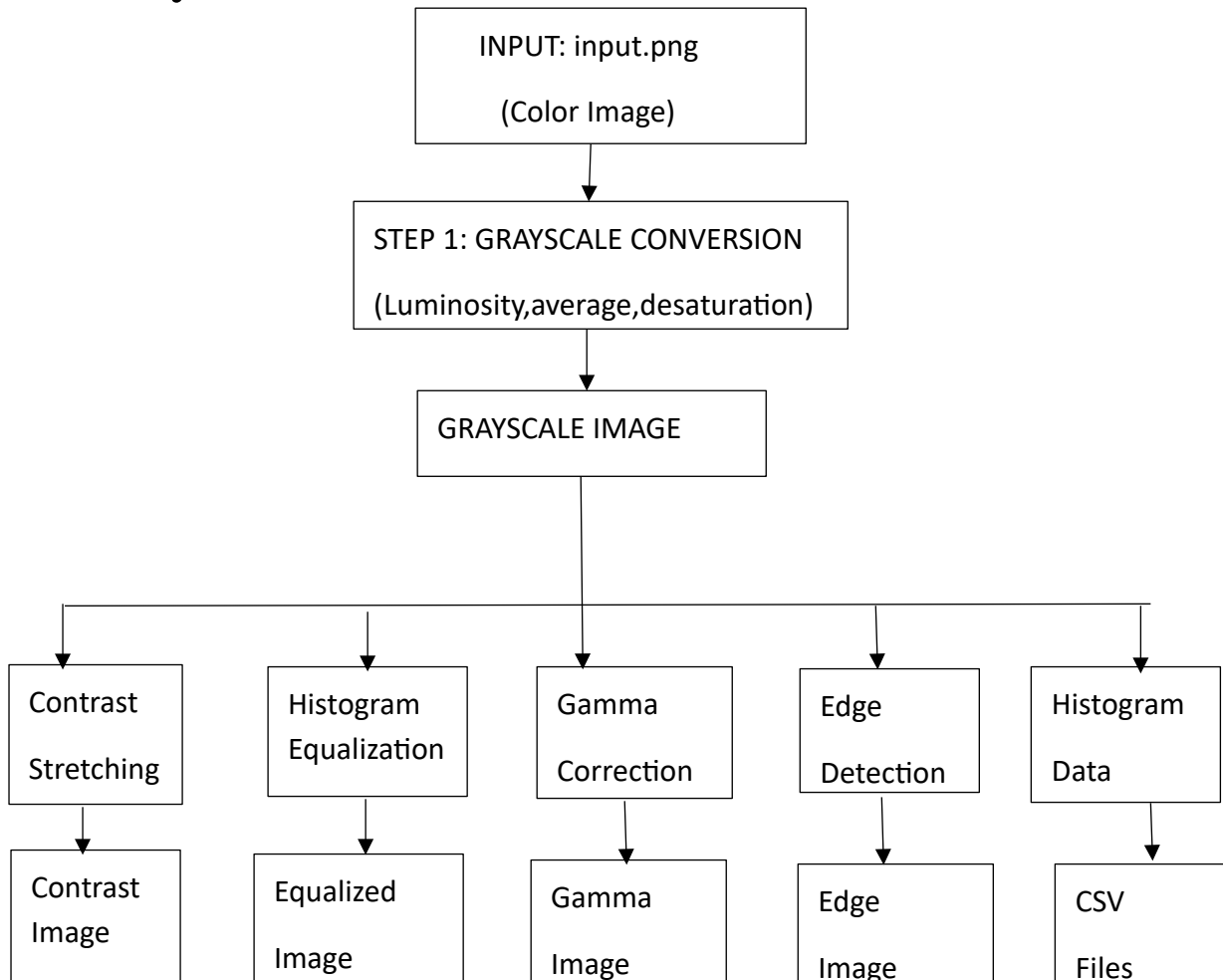
$G_x =$

1	0	-1
2	0	-2
1	0	-1

$G_y =$

1	2	1
0	0	0
-1	-2	-1

3. System Architecture



4 Results and Analysis

This system generates multiple images.

Input image



Output images

Average grayscale image



Luminosity Grayscale image



Desaturation grayscale image



Contrast image



Equalized image



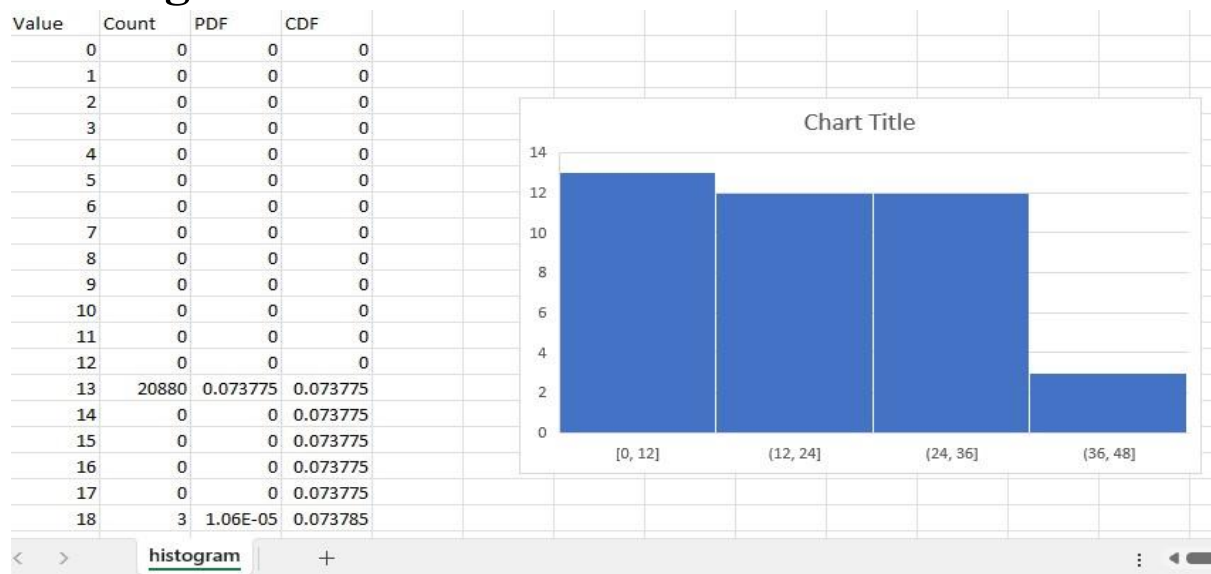
Gamma image



Edges Image



5. Histogram data



Conclusion

This project demonstrates a Grayscale image analysis and Enhancement system. The goal is to output or show multiple grayscale conversion methods, histogram analysis and equalization, image enhancement with contrast stretching, gamma correction, edge detection using sobel kernel convolution and save the histogram data.

6. Appendix - Source Code

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <algorithm> #include
"Image.h"
using namespace std;

// Grayscale Methods
unsigned char grayAverage(unsigned char r, unsigned char g, unsigned char b) {
return (r + g + b) / 3;
}
unsigned char grayLuminosity(unsigned char r, unsigned char g, unsigned char b) {
return (unsigned char)(0.2126 * r + 0.7152 * g + 0.0722 * b);
}
unsigned char grayDesaturation(unsigned char r, unsigned char g, unsigned char b) {

    unsigned char maxv = max(r, max(g, b));
    unsigned char minv = min(r, min(g, b));    return
(maxv + minv) / 2;
}
// Gamma Correction
unsigned char gammaCorrection(unsigned char v, float gamma) {

    return (unsigned char)(255 * pow(v / 255.0, gamma));
}
int main() {    ColorImage
img;
```

```

img.Load("input.png");
int w = img.GetWidth();
int h = img.GetHeight();

int choice;
cout << "GRAYSCALE IMAGE SYSTEM" << endl;
cout << "1 - Average Grayscale" << endl; cout
<< "2 - Luminosity Grayscale" << endl; cout <<
"3 - Desaturation Grayscale" << endl; cout <<
"4 - Histogram Equalization" << endl; cout <<
"5 - Contrast Stretching" << endl; cout << "6 -
Gamma Correction" << endl; cout << "7 -
Edge Detection" << endl; cout << "Choose: ";

cin >> choice; // 1. Average
Grayscale if (choice == 1) { for
(int y = 0; y < h; y++) { for (int
x = 0; x < w; x++) { RGBA p =
img(x, y); unsigned char g =
grayAverage(p.r, p.g, p.b);
img(x, y).r = g;
img(x, y).g = g;
img(x, y).b = g;
}
}
img.Save("average.png"); cout
<< "Saved average.png" << endl;
}
// 2. Luminosity Grayscale
else if (choice == 2) {
for (int y = 0; y < h; y++) { for
(int x = 0; x < w; x++) { RGBA p
= img(x, y); unsigned char g =
grayLuminosity(p.r, p.g, p.b);
img(x, y).r = g;
img(x, y).g = g;
img(x, y).b = g;
}
}
img.Save("luminosity.png");
cout << "luminosity.png saved" << endl;
}
// 3. Desaturation Grayscale else if
(choice == 3) { for (int y = 0; y < h;
y++) { for (int x = 0; x < w; x++) {
RGBA p = img(x, y); unsigned
char g = grayDesaturation(p.r,
p.g, p.b);
img(x, y).r = g;
img(x, y).g = g;
img(x, y).b = g;
}
}
img.Save("desaturation.png");
cout << "desaturation.png has been saved" << endl;
}
// 4. Histogram Equalization else if
(choice == 4) { int histogram[256] =
{}; float pdf[256] = {}; float
cdf[256] = {}; int equalized[256] =
{}; for (int y = 0; y < h; y++) {
for (int x = 0; x < w; x++) {
RGBA p = img(x, y); unsigned

```

```

char g =          grayLuminosity(p.r,
p.g, p.b);
    img(x, y).r = g;
img(x, y).g = g;
img(x, y).b = g;
    histogram[g]++;
    }
    }
    int pixels = w * h;    for
(int i = 0; i < 256; i++) {
pdf[i] =
    histogram[i] / (float)pixels;
    }
    cdf[0] = pdf[0];    for
(int i = 1; i < 256; i++) {
cdf[i] =
    cdf[i - 1] + pdf[i];
    }
    for (int i = 0; i < 256; i++) {

        equalized[i] =
(int)(255 * cdf[i]);
    }
    for (int y = 0; y < h; y++) {
for (int x = 0; x < w; x++) {
int v = img(x, y).r;
img(x, y).r = equalized[v];
img(x, y).g = equalized[v];
img(x, y).b = equalized[v];
    }
    }
    img.Save("equalized.png");
ofstream file("histogram.csv");
file << "Value,Count,PDF,CDF\n";
for (int i = 0; i < 256; i++) {    file
<< i << ", "    << histogram[i]
<< ", "
        << pdf[i] << ", "
        << cdf[i] << "\n";
    }

    file.close();
cout << "Saved equalized.png" << endl;
cout << "Saved histogram.csv" << endl;
}

// 5. Contrast Stretching else if
(choice == 5) {    int minv = 255;
int maxv = 0;    for (int y = 0; y < h;
y++) {    for (int x = 0; x < w; x++) {
RGBA p = img(x, y);    unsigned
char g =          grayLuminosity(p.r,
p.g, p.b);    img(x, y).r = g;
img(x, y).g = g;    img(x, y).b = g;

        if (g < minv) minv = g;
if (g > maxv) maxv = g;
    }
    }
    for (int y = 0; y < h; y++) {    for (int
x = 0; x < w; x++) {    int v = img(x,
y).r;    v = (v - minv) * 255 / (maxv -
minv);

```

```

        img(x, y).r = v;
img(x, y).g = v;
img(x, y).b = v;
    }
}
img.Save("contrast.png");
cout << "Saved contrast.png" << endl;
}
// 6. Gamma Correction else if (choice
== 6) { float gamma = 0.8f; for
(int y = 0; y < h; y++) { for (int x = 0;
x < w; x++) { RGBA p = img(x, y);
unsigned char g =
grayLuminosity(p.r, p.g, p.b); g =
gammaCorrection(g, gamma);

        img(x, y).r = g;
img(x, y).g = g;
img(x, y).b = g;
    }
}
img.Save("gamma.png");
cout << "Saved gamma.png" << endl;
}
// 7. Edge Detection
else if (choice == 7) {
ColorImage temp = img;
int Gx[3][3] = { {-1,0,1},
{-2,0,2},
{-1,0,1}
};
int Gy[3][3] = { {-1,-2,-1},
{0,0,0},
{1,2,1}
};
for (int y = 0; y < h; y++) { for
(int x = 0; x < w; x++) { RGBA p
= img(x, y); unsigned char g =
grayLuminosity(p.r, p.g, p.b);

        temp(x, y).r = g;
    }
}
for (int y = 1; y < h - 1; y++) {
for (int x = 1; x < w - 1; x++) {
    int sx = 0; int sy =
0; for (int j = -1; j <= 1; j++)
{ for (int i = -1; i <= 1; i++)
{
        int pixel
=
temp(x + i, y + j).r;
sx += pixel *
Gx[j + 1][i + 1];
sy += pixel *
Gy[j + 1][i + 1];
    }
}
int value =
sqrt(sx * sx + sy * sy);

```

```
        if (value > 255)
value = 255;          img(x,
y).r = value;        img(x,
y).g = value;        img(x,
y).b = value;
    }
    }
    img.Save("edges.png");
    cout << "Saved edges.png" << endl;
}
else {
    cout << "option not found" << endl;
}
return 0;
}
```